Concordia University
Gina Cody School of Engineering and Computer Science

COMP 472 - Introduction to Artificial Intelligence
Final Project Report
Image Classification CIFAR-10

Dominique Isbister, 40210056
Mohamed Mahmoud, 40283160
Kenny Luo-Li, 40237402

Submitted to Kefeya Qaddoum
On November 25th 2024

Github Repository Link

You can find the implementation of this project, including the models, their training, and evaluations, in the repository linked here.

Model Architectures and Training

In order to classify images on the CIFAR-10 database, we have used four distinctive AI models; Naive Bayes, decision trees, multi-layer perceptrons and convolutional neural networks. For each model we implemented different variations allowing us to thoroughly compare the performance of each model. The evaluation methods used to compare performance were accuracy, recall, precision and F1 score as well as confusion matrices generated to visually assess the performance. We implemented the four models mentioned above with following variants; the Naive Bayes and decision trees were implemented using sklearn libraries and a custom implementation.The decision trees were also implemented with varying depths of 25, 50 and 75 giving us a total of 6 variants for the decision tree model. The multi-layer perceptrons were implemented with different configurations such as 3 layers, 4 layers, hidden layers of small and large sizes for a total of four variants. The last model was convolutional neural network for which we implemented four different variants, the basic VGG11 architecture, a variant with an additional convolutional layer, one with a larger kernel in the first convolutional layer and a variant with global average pooling as opposed to connected layers. Coming to a total of 16 models implemented, this includes base models and their variants.

In order to train the data we began by adapting the size of the images to 224x224 pixel tensors in order to fit the model input requirements. The training data constitutes a subset of the CIFAR-10 dataset, selecting 500 images for training and 100 test images per class for each of the 10 object classes. In other words, 5000 images to train and 1000 to test. Using the preset ResNet-18 model, we extracted 512 features from the images in the dataset. Applying principal component analysis (PCA) allowed us to reduce the amount of features to 50. Once our data and models were ready, the training process could start. To train both the multi-layer perceptrons and CNN we ran 10 epochs and loss was calculated using CrossEntropyLoss. Additionally, we optimized both the CNN and multi-layer models using SGD. The decision to use SGD originated from the need to better generalize the unseen data, we found SGD to distinguish itself as a strong generalization optimizing method as it explores a wider space of flatter minima. Training the data with the previous criteria led us to adequate results which we evaluated using the evaluation methods mentioned previously.

Evaluation of the System

After implementing our 16 different models and variants, we ensured to evaluate them adequately in order to assess performance. To do so, we used the following five evaluation methods; accuracy, recall, precision and F1 score calculations. Additionally, creating confusion matrix graphs to visually assess performance.

Table 1: Results for accuracy, recall, precision and F1 for four models and variants

Models	**Accuracy**	**Recall**	**Precision**	**F1 score**
Naive Bayes (scratch)	0.7980	0.7980	0.802085	0.798307
Naive Bayes (sklearn)	0.7980	0.7980	0.802085	0.798307
Decision Tree (scratch 25)	0.5860	0.5860	0.589316	0.586323
Decision Tree (scratch 50)	0.5860	0.5860	0.589316	0.586323
Decision Tree (scratch 75)	0.5860	0.5860	0.589316	0.586323
Decision Tree (sklearn 25)	0.5930	0.5930	0.592175	0.592187
Decision Tree (sklearn 50)	0.5850	0.5850	0.586143	0.584608
Decision Tree (sklearn 75)	0.5880	0.5880	0.590410	0.588213
Multi-layer (3)	0.8250	0.8250	0.828410	0.824802
Multi-layer (4)	0.8160	0.8160	0.820775	0.816616
Multi-layer (hidden small)	0.8110	0.8110	0.814525	0.811328
Multi-layer (hidden large)	0.8220	0.8220	0.827853	0.823097
CNN (base)	0.8264	0.8264	0.830704	0.826448
CNN (additional layer)	0.7855	0.7855	0.792468	0.781755
CNN (larger kernel)	0.8129	0.8129	0.820470	0.813941
CNN (global pooling)	0.8087	0.8087	0.813342	0.808302

When observing these results, we were able to highlight some patterns. Each evaluation method provides us valuable insights for the performance of our models.

For starters, the naives bayes methods performed similarly in terms of results, both the scratch and sklearn variants coming to the same results for all four evaluation methods. We also notice true positives (recall = 0.7980) and precision of results (precision = 0.802085) are fairly high indicating the model is more often than not correct about its predictions, making it rather trustworthy.

On the other hand, the decision trees performed quite worse compared to the naive bayes models. Observing the sklearn variants to be slightly different in terms of results compared to the models from scratch falling at 0.5860 for accuracy and recall, 0.589316 for precision and 0.586323 for F1 score for all depths tested. The sklearn variants results were between 0.5850 and 0.5930 for accuracy and recall, 0.586143 and 0.592175 for precision and 0.584608 and 0.592187 for F1 scores. For sklearn variants, the smallest depth performed best, with largest depth coming in second and depth of 50 last, but the difference between evaluation results were very minor. Concluding that depth of 25 would be best to use, naive bayes does take the lead in terms of performance up until now.

With accuracy and precision ranging from 0.8110 to 0.8250, 0.814525 to 0.828410 for precision and 0.811328 to 0.824802 for F1 score the multi-layer perceptrons proves to be the most trustworthy model evaluated up until now, specifically three-layer perceptron. To be noted that, modifying the model by adding layers or using hidden layers slightly affects performance negatively.

Lastly, at a glance the convolutional neural network model performs best. With VGG11 architecture the accuracy and recall are evaluated at 0.8264, precision at 0.830704 and F1 at 0.826448, being the best performing model we implemented. However some variants significantly impact performance such as adding a convolutional layer in the last block bringing performance down to below even the Naive Bayes model. Any other CNN variants also prove to be non-desirable as performance decreases significantly enough that models such as multi-layer perceptrons are better performing than CNN variants.

To conclude, the best performing model we implement for the CIFAR-10 image dataset is the base convolutional neural network model, but the multi-layer perceptron model and its variants prove to be acceptable choices as ML models in this situation.

Found below are confusion matrices for all 16 models we implemented. Observing these confusion matrices allows us to understand the evaluation results better.

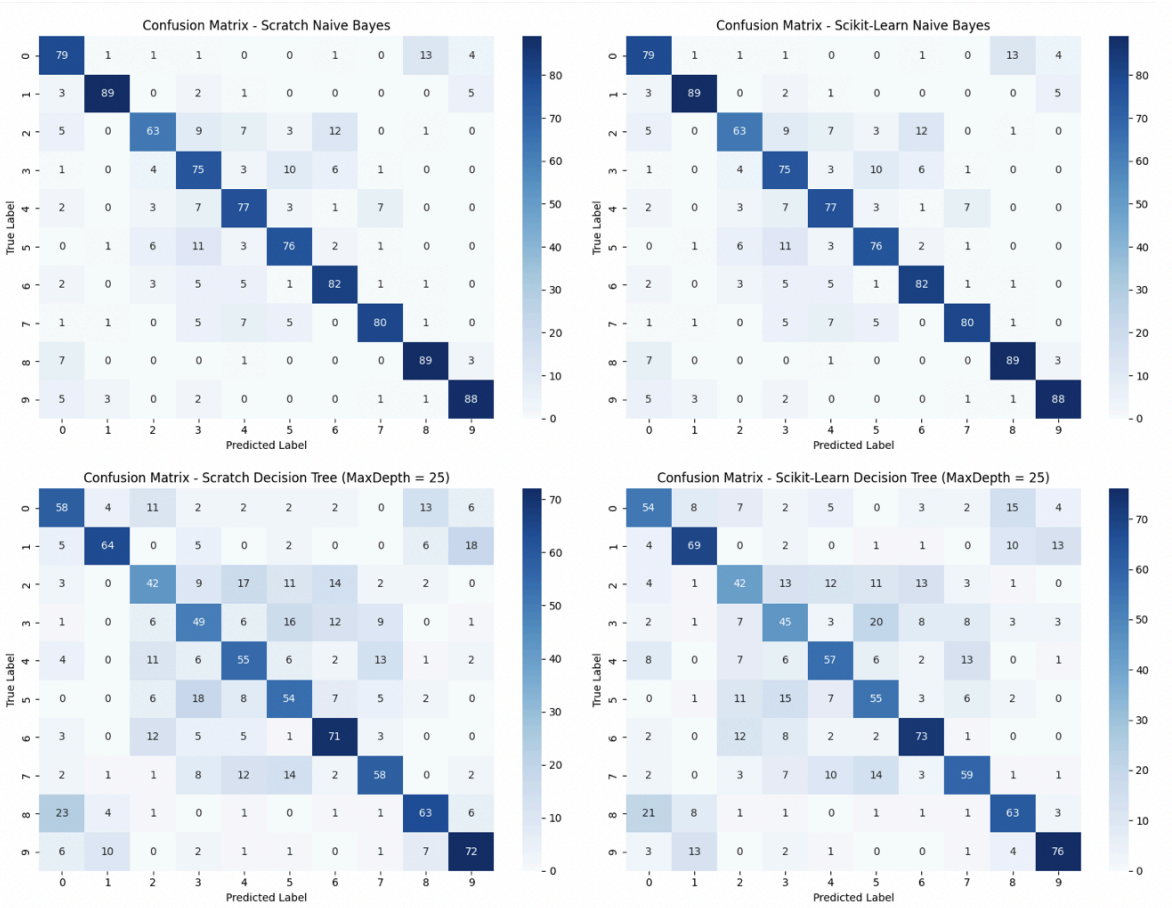

Image 1: Confusion matrix for Naive Bayes and Decision Trees depth 25

The first two graphs representing the Naive Bayes model do not alarmingly confuse any classes, but if any do get confused they are most often class 0 being labeled as class 8, class 2 as 6, class 3 as 5 and class 5 as 3. Naive Bayes is a fast model that can make predictions on data of high dimensions, but it does assume all features are independent from each other, which might not be the case. Image classifications depend on color, shape, pixel dependencies, etc. The confusion between classes 3 and 5 might be caused by the assumption of independence, possibly due to pixel dependencies of other dependent features. Similarly classes 0 and 8, and 2 and 6 might be of similar shapes or color, thus why the model is confusing them. Overall the Naive Bayes model did rather well at classifying the CIFAR-10 images and this might be due to the 512

features being cut down to 50, the 50 remaining features may have been contrasting features that were rather independent to begin with.

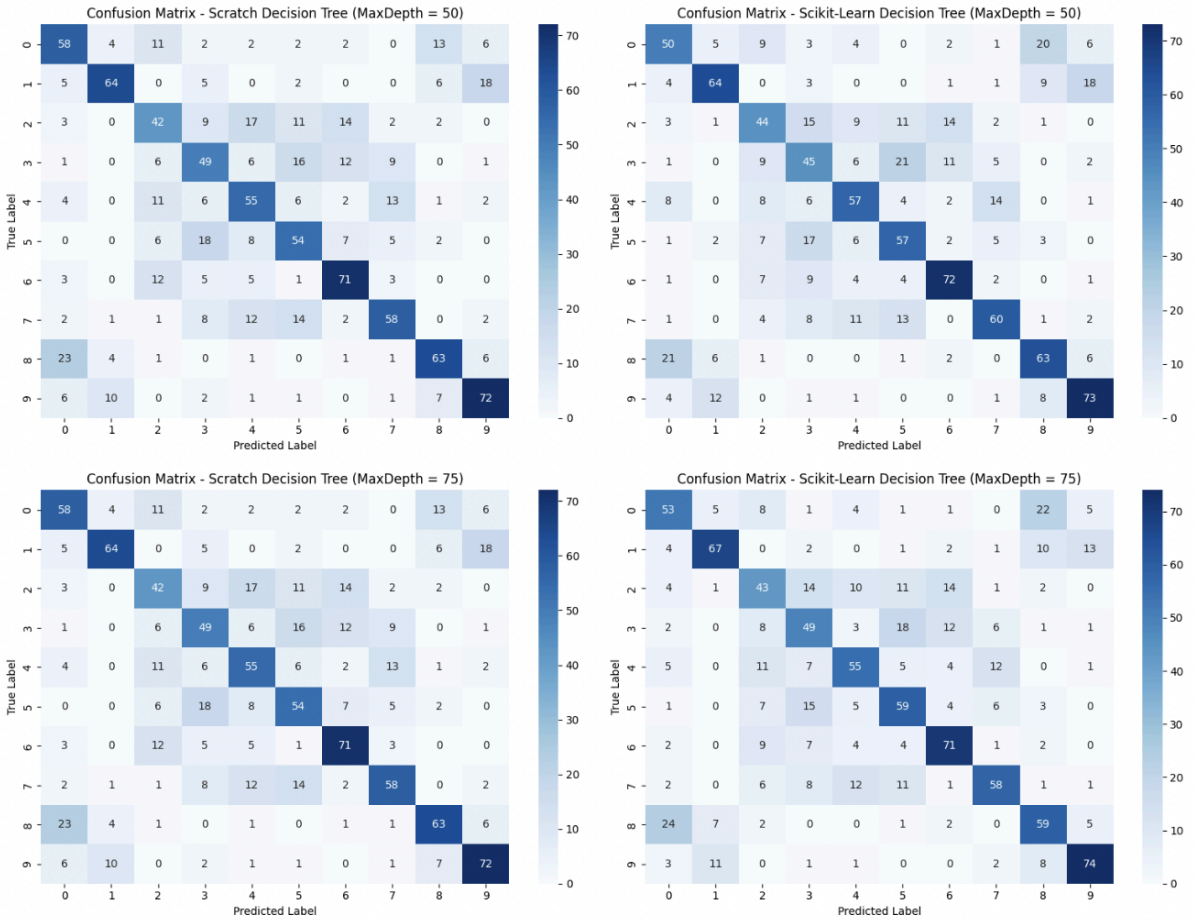

Image 2: Confusion matrix for Decision Trees depth 50 and 75

The decision tree model underperforms when it comes to classifying class 2, mistaking it for multiple classes such as class 4, 5 or even 6. It also misses the mark for class 3, miss-labeling it as a class 5, 6 or 7. A rather high confusion level is also the classification of class 8 as a class 0, always being just above 20%. The decision tree approach is known for its ability to detect various relationships in the data, but it does struggle with non-linear relationships and overfitting when the depth is too deep or the dataset is too noisy. The miss-classification of label 2 might be due to the similarity in features of label 2 to classes 4, 5 and 6, thus possibly causing some overfitting. The same thing might be happening for label 3 and its similarities to label 5, 6 and 7. The miss-labeling of label 8 as a label 0 might be due to noisy data that is being overfit into label 0. Similarly to the naive bayes model, the correctly classified classes may have simply been linearly related, with high contrasting relationships allowing some sort of precision from the decision tree model.

The performance in terms of depths was best at 25, this may be due to the fact that overfitting was avoided, while some distinguishing factors were captured. Depth 50 decreased in performance, it is possible that the added depth allowed it to capture noisy features and specific patterns, thus decreasing performance. The later increase in performance at depth 75 with a remaining lower performance compared to depth 25 indicates overfitting. To conclude, increasing depth for the decision trees model may have allowed to capture distinguishing features, but lead to overfitting as well.

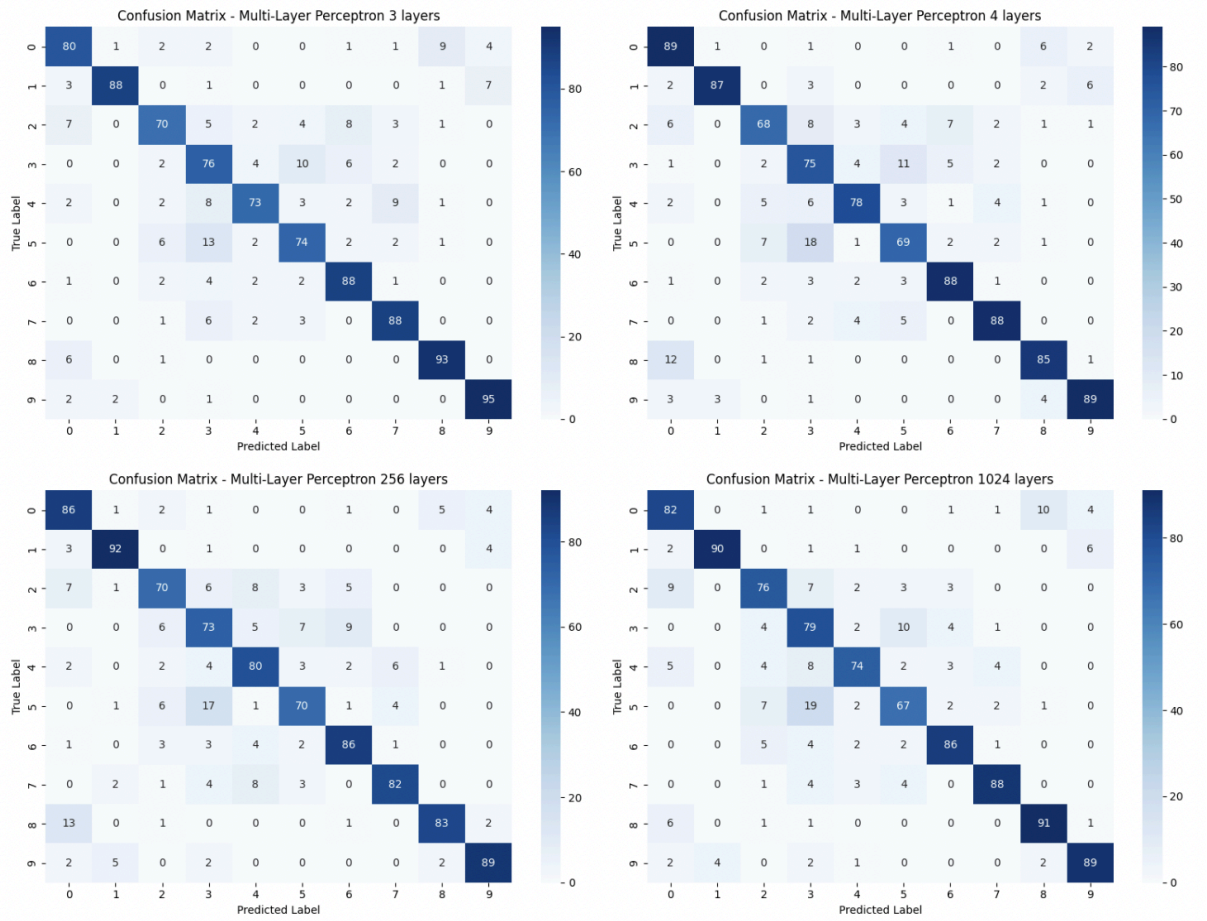

Image 3: Confusion matrix for Multi-Layer Perceptrons and variants

Similarly to the Naive Bayes model, there are no alarming confusion rates found for the multi-layer perceptron model, but any source of miss-labeling most likely originates from class 5 being labeled class 3 as it is above 10% in all variants. Multi-layer perceptrons are known for image recognition as they are good for nonlinear problems, but similarly to decision trees, this model can overfit if the provided data isn't adequate. Once again the classes 3 and 5 are being confused by an ML model, this might be due to similar colors or patterns as its independence and non-linearity are distinguishable by the multi-layer perceptron model. We also notice a slightly higher confusion rate when labeling class 8 as a class 0, as a model that is also prone to

overfitting, it is possible that similarly to the decision tree model, the multi-layer has found the noisy data and has begun overfitting to it. In general the confusion rates are much smaller for the multi-layer perceptrons and this is probably due to the non-linearity of the model allowing it to recognize the unique features found in the CIFAR-10 dataset, something the previous two models were not capable of doing.

In terms of depth, adding an extra layer to the multi-layer model caused a decrease in performance which also indicates the start of overfitting, probably caused by noisy data as discussed previously. The variant in which we increased layer size allowed the model to once again pick up on more data, but decreased performance which indicates noisy data and overfitting. On the other hand, the variant in which we had smaller hidden layers also decreased in performance, but this could be due to insufficient data being fed to the model, leading to inaccurate results. Additionally, with a larger kernel the model performance decreased we assume that it began overfitting or labeling noisy data. To conclude, the 3-layer model performed best as it was balanced between having enough depth and data without taking noisy data into account or overfitting.

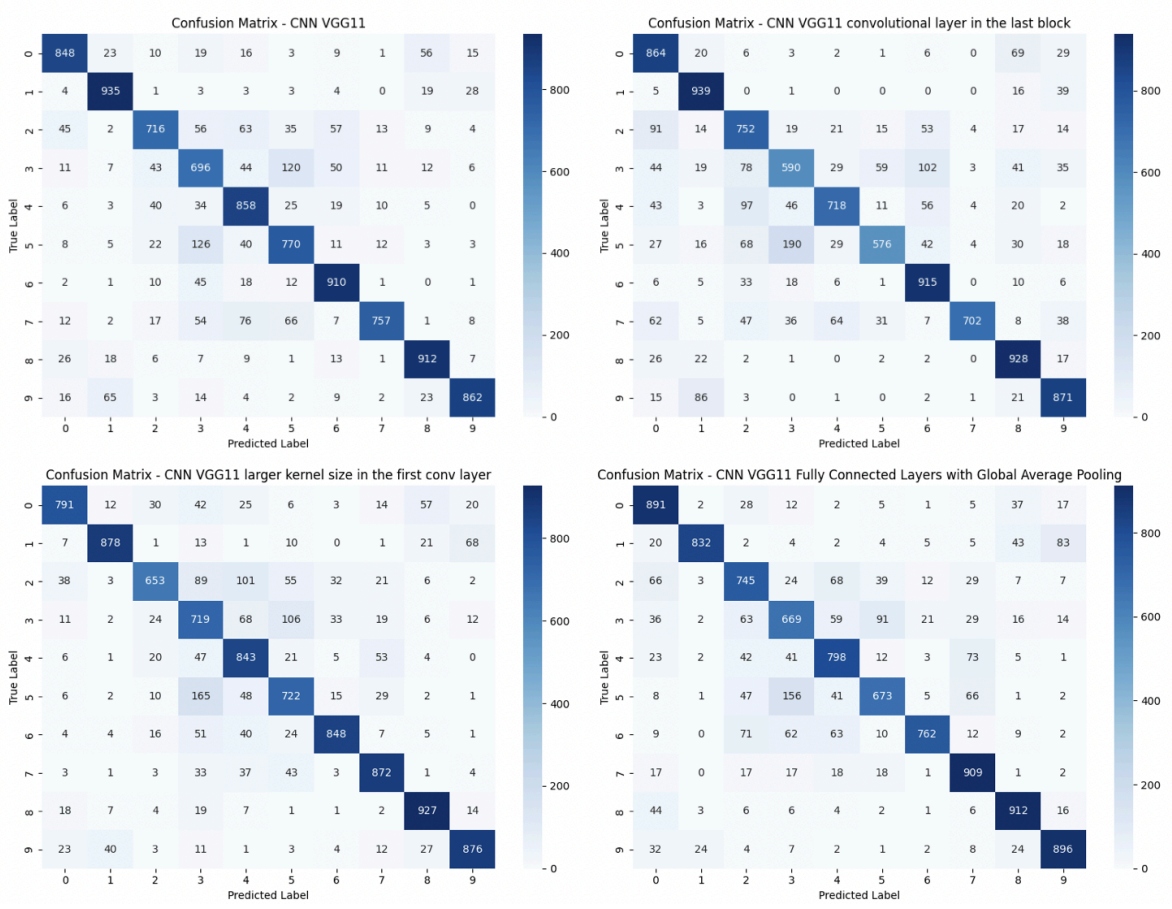

Image 4: Confusion matrix for CNN and variants

Once again, there are no alarming confusion rates found for the CNN model, but we do notice the difference between the base CNN and its variants. Although the highest confusion rate being found at true label 5 predicted as a label 3, the distribution of the confusion rates are quite different in each of the four variants. Compared to other models where high confusion rates were repetitive from one variant to another so much so that a pattern within the graph is visually noticeable, these CNN models differ significantly from each other. CNN models are known for flexibility in learning which makes it particularly ideal for image classification problems, but does tend to need larger amounts of data. The confusion between class 5 and 3, a confusion many of the other models encountered as well, in this particular model could be due to insufficient data volume, preventing the model from picking up on a pattern. Overall the CNN model performed quite well, all classes are well organized, with confusion index being near or below 10% in majority. One could consider the success of the CNN model to be due to its spatial awareness making it the best ML model when classifying images found in the CIFAR-10. Diving deeper into the different variants, when adding an additional layer or increasing the depth, it allowed the model to capture different patterns in the data, but seemed to cause overfitting due to performance decrease in both the calculated evaluation methods and the confusion matrix. When increasing the kernel size, the performance of the model decreased as well, we speculate this to have happened due to the broader focus which prevented the model from being able to analyze precise classification. The base kernel size seemed to have been a perfect balance between too little and too much focus of the data.

To conclude our analysis between the Naive Bayes, decision trees, multi-layer perceptrons and CNN models, we shall highlight an overall best performing model. Considering the evaluation results being highest for the CNN base model, the confusion matrix being most linear for either the 3-layer perceptron or the CNN base models and the convolutional neural network model's structured layers and feature extraction capabilities, the CNN base model distinguishes itself as most efficient model for the CIFAR-10 dataset. A close upcomer is undoubtedly the multi-layer perceptron model and its variant as opposed to the CNN model, the multi-layer variants performed quite well. With multi-layer (3 layers) coming in second best and multi-layer (large hidden layers) coming in third place. While the Naive Bayes model did not perform underwhelmingly, it was overshadowed by other models we implemented and decision trees being left dead last considering its assumptions of independence and linear relationships, making it unfit for image classification. In other words, the Naive Bayes model showed consistent, but limited results and the decision trees model was prone to overfitting, especially at deeper depths. An important takeaway from this project is the importance of the choice of a machine learning model compared to the data at hand. No matter the quality of implementation, an unfit model for a specific dataset will underperform.

References

[1] "Feature extraction," *scikit-learn*, 2024. [Online]. Available: https://scikit-learn.org/1.5/modules/feature_extraction.html. [Accessed: 4-Oct-2024].

[2] "ML | Naive Bayes Scratch Implementation using Python," *GeeksforGeeks*, [Online]. Available: https://www.geeksforgeeks.org/ml-naive-bayes-scratch-implementation-using-python/. [Accessed: 6-Oct-2024].

[3] A. Raj, "Build VGG Net from Scratch with Python," *Analytics Vidhya*, 29-Jun-2021. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/06/build-vgg-net-from-scratch-with-python/. [Accessed: 02-Nov-2024].

[4] A. S. Scribe, "Building a Neural Network from Scratch in Python," *Medium*, 06-Feb-2021. [Online]. Available: https://medium.com/@aisagescribe/building-a-neural-network-from-scratch-in-python-b9cbd5cdfce6. [Accessed: 12-Nov-2024].

[5] "Decision Tree Implementation in Python," *GeeksforGeeks*, [Online]. Available: https://www.geeksforgeeks.org/decision-tree-implementation-python/. [Accessed: 14-Nov-2024].

[6] "What is Overfitting?" *Amazon Web Services (AWS)*, [Online]. Available: https://aws.amazon.com/what-is/overfitting/. [Accessed: 16-Nov-2024].